



APRIL 1, 2024

TERM ASSIGNMENT REPORT

CSCI 4145 – CLOUD COMPUTING

MANAR ELGAMIL
B00878259



Table of Contents

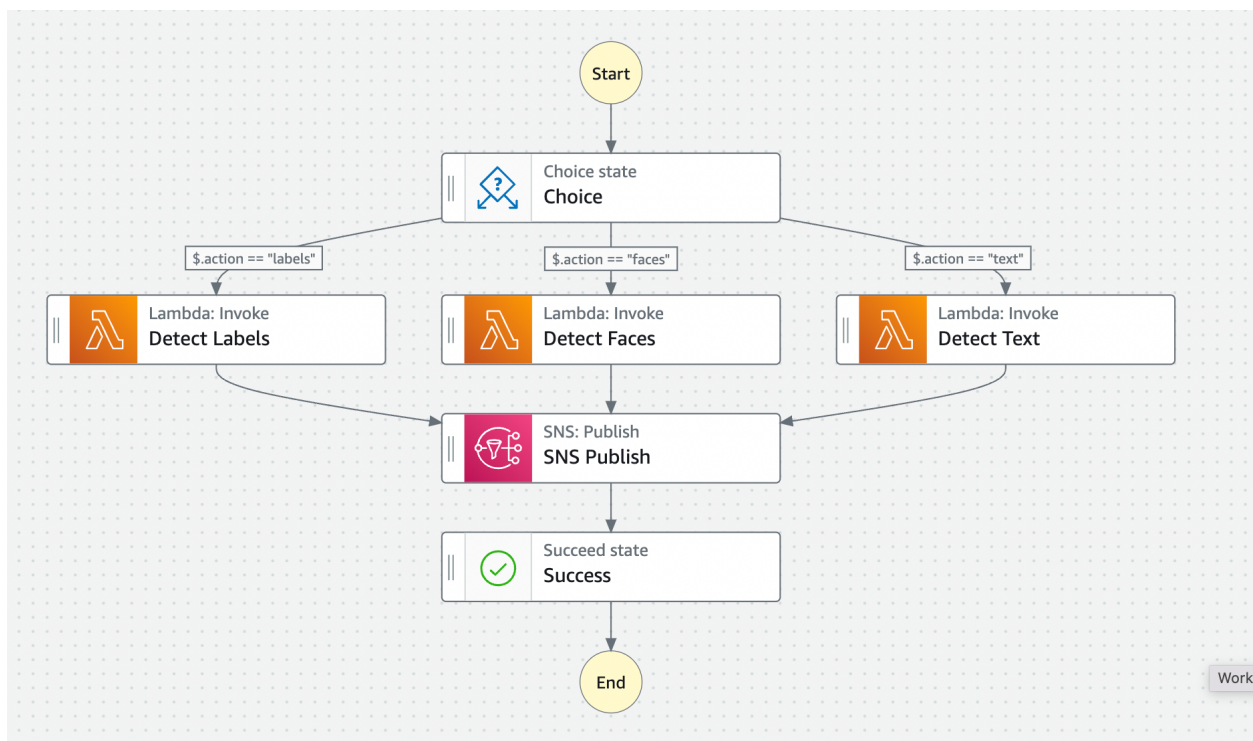
Section 1: What I Built and its Purpose.	2
What did I build?	2
What is its Purpose (use cases)?.....	5
Section 2: Requirement Satisfaction	6
Compute (2)	6
Storage (1)	6
Network (1).....	6
General (2).....	6
Section 3: Deployment Model	7
Section 4: Delivery Model	8
Section 5: Architecture	9
Section 6: Security	11
Data in Transit	11
Data at Rest	11
Section 7: Cost-Metrics	13
Pricing of System Components	13
Total System Price	14
Hardware	15
Software.....	16
Rough Estimate	16
Section 10: Application Evolution.....	17

Section 1: What I Built and its Purpose.

What did I build?

My project is an image processing program composed of a state machine, that combines Lambda Functions, AWS SNS to process image and forecast the result to a list of users. In order to trigger the program, a POST request is sent to a specified API. This request includes the image name and the type of processing that the user wants as inputs to the step functions. After the processing is complete, the results are broadcast to the list of subscribers.

Here is the design my **step function** (There is a picture of my completed Architecture later):



The 3 separate AWS Lambda functions do different kind of processing to the specified image, the user choose that type of processing in the “action” attribute that they specify in the input when sending their POST request. There are 3 different actions:

- 1- **Labels:** This analyzes images by labelling the data in them, it analyzes how many people are in an image, trees, boards, chairs - basically object in an image.
- 2- **Faces:** This detects faces and analyzes them. An example is the whether the detected face is smiling; or if it is wearing sunglasses; it's age range; and its gender, and more attributes.

- 3- **Text:** This detects any text in an image, it could be very useful for financial institutions especially. Because it allows them to detect the text in images.

All of this processing is executed by AWS Rekognition, AWS Lambda connects to AWS Rekognition by sending a request to its API with the S3 bucket name and image name, which they get from the user, AWS Rekognition then returns a response of the processing results.

This is the code for the **DetectFaces Lambda Function**:

```
import json
import boto3

client = boto3.client('rekognition')

def lambda_handler(event, context):

    bucket = "picturesssss-bucket"
    file = event['filename']

    #This is request sent to AWS Rekognition API, with the S3 bucket and image name
    response = client.detect_faces(Image={'S3Object': {'Bucket': bucket, 'Name':
file}}, Attributes=['ALL'])

    output = ""

    for face in response['FaceDetails']:
        output += json.dumps(face, indent=2)

    if output == "":
        return "No Faces found!"
    else:

        return output
```

The other Lambda functions are similar in implementation, the main difference is the different methods that is used when sending the request to AWS Rekognition. After the Lambda function receives the Rekognition's response, it returns this response back, which is the output of the

function. This output is then used as input for AWS SNS which broadcasts this response to the list of subscribers.

This is the **API** that the user must post to:

<https://t8txrx5i09.execute-api.us-east-1.amazonaws.com/dev/execution>

This API was created using API Gateway, and only accepts post requests.

This is the format of the JSON that should be present in the body of the POST request:

```
{
  "input": "{\"filename\": \"<image name> \", \"action\": \"<action goes here, user can choose 'labels', 'faces' or 'text'> \"}\",
  "name": "<OPTIONAL: Here you can add the execution name, and it must be a new, unrepeated name, but it is an optional field, you don't need to specify it.>",
  "stateMachineArn": "<Here I add the arn of my statemachine, this needs to be identical to the example below>"
}
```

Here is a sample JSON in the body of the POST request:

```
{
  "input": "{\"filename\": \"no-smile-no-sunglasses.jpg\", \"action\": \"faces\"}\",
  "name": "MyExecution8",
  "stateMachineArn": "arn:aws:states:us-east-1:590183949889:stateMachine:final-proj-machine"
}
```

What is its Purpose (use cases)?

This program has many use cases. We will discuss 3 here (one for each action), but it could be adapted to suit many other cases that requires image processing.

First example: A team of doctors specializing in analyzing patient scans, and they are collaborating on a project together. They are interested in analyzing patients scans, but these scans take time and are very detailed, also every doctor is working with a different patient, but they all need to collaborate and share their results simultaneously. The doctors would like the analysis to be automated and they also want to be notified of the analysis of every time a new scan is added to their collection. This would be a very suitable application/ use case for my program. The prerequisites that need to occur before this could be implemented is that the group of doctors' email would be added to the SNS subscriber's list. The reason this step cannot be automated is because users would need to confirm their emails before the messages could be broadcast to them. Additionally, the doctor that need to upload their patient's scans to the S3 bucket and then send the POST request with the scan's name (picture-name / filename), with the "label" action, to trigger my program. Then my program will commence its execution and upon successful completion it will broadcast the results of the analysis of the image to all the doctors in the subscriber's list. My program should be excellent in analyzing images, with as much automation as much automation as possible. (Fewer user interactions)

Second example: Lawyers and accountants could benefit from being able to extract text from pictures of important documents. They would have to have the same prerequisites as mentioned above, the team of lawyers or accountants would have to be subscribed to the topic, and whenever someone adds a picture to the S3 bucket, they would have to send the POST request to trigger the program's execution.

Third example: Finally, governments and policemen could use it to analyze scenes, like camera surveillance videos or pictures. AWS Rekognition can identify and label the objects in pictures. For example, if there is a picture of a crime scene, it can identify the tool used for murder and objects that were present there. Governments could use it to analyze the times with the highest traffic, by counting how many cars are in the pictures. They can use it to gain statistics about the population demographics through these. It could also be used to identify criminals in a bigger crowd using the detect faces feature. (Just like before prerequisites would have to be set up before they can start using the program)

Section 2: Requirement Satisfaction

Compute (2)

AWS Lambda – Lambda allows you to run code without provisioning or managing a server, and it scales automatically (Amazon, 2024). The alternative would be to set up an EC2 server, which would require the developer to manage and maintain the server and create the scaling logic. Additionally, in Lambda you only get charged based on the number of requests and their runtime, since requests are unexpected, you would have to run the EC2 instance 24/7, which is more expensive.

AWS Step Functions – This allowed me to easily connect and coordinate the components of my application and provides a useful visual workflow. Suits my serverless model, since they don't require server management, and scales automatically. There are no other service that perform this function except AWS Step Functions.

Storage (1)

AWS S3 - Since my application's storage purpose is very simple (just storing images), S3 becomes the perfect option because it's optimized for files, unlike AWS Dynamo DB which has a maximum item size 400 KB (Amazon, 2024) so it will limit the images resolution or AWS Aurora which is optimized for storing complex transactional workloads, so using it to store image would be pointless because it's designed to perform more complex operation. Additionally, Dynamo DB and Aurora are both very expensive compared to S3.

Network (1)

AWS API Gateway – are used to manage APIs, which is a key component in my design, and they easily integrate with my step function. AWS VPC would not be a good option because it has no API interface, and it would add an unnecessary level of complexity to my application. Amazon EventBridge is also not suitable for my application because it's designed to route events between AWS services or third-party integrations, but not to handle the creation of APIs.

General (2)

AWS Rekognition – Allows you to analyze images and videos without creating machine learning models or algorithms. According to my knowledge AWS does not have any alternatives for this service.

AWS SNS – Allows you to send emails/text messages to clients of endpoints subscribed to a topic. This is very suitable for my application because it allows me to notify the users working on the same project of the results, once the image processing is completed. An alternative is using AWS SQS which is used for message queuing, it puts great emphasis on the message being sent in the correct order of the queue (FIFO queue). However, the order of the messages is not important for my application, I just need all the users to be notified. Therefore, AWS SNS would be the better option.

Section 3: Deployment Model

The deployment model that I have picked for my program is **public cloud**. The public cloud model involves using cloud services that are publicly accessible and owned by third-party cloud providers, in our case its AWS. It's an ideal deployment model for my image processing software because it offers scalability, a pay-as-you-go pricing structure, and a many integrated services that facilitate rapid deployment of the program. It allows handling different workloads efficiently—scaling resources up or down as needed—without incurring the costs and complexities of maintaining physical infrastructure.

Its way cheaper when compared to other models like private cloud, which would require a huge investment in hardware and maintenance, and it lacks the scalability of the public cloud. A hybrid cloud could be beneficial for using both on-premises and cloud resources, but it introduces an unnecessary level of complexity that my application doesn't need. A community cloud doesn't provide the same level of global reach and service breadth, and a multi-cloud strategy would involve managing across different platforms, which can complicate operations without any benefits to my use case.

Therefore, the public cloud is the most suitable as it aligns with the need for a flexible, cost-effective, and easy-to-manage environment that meets all my application's needs.

Section 4: Delivery Model

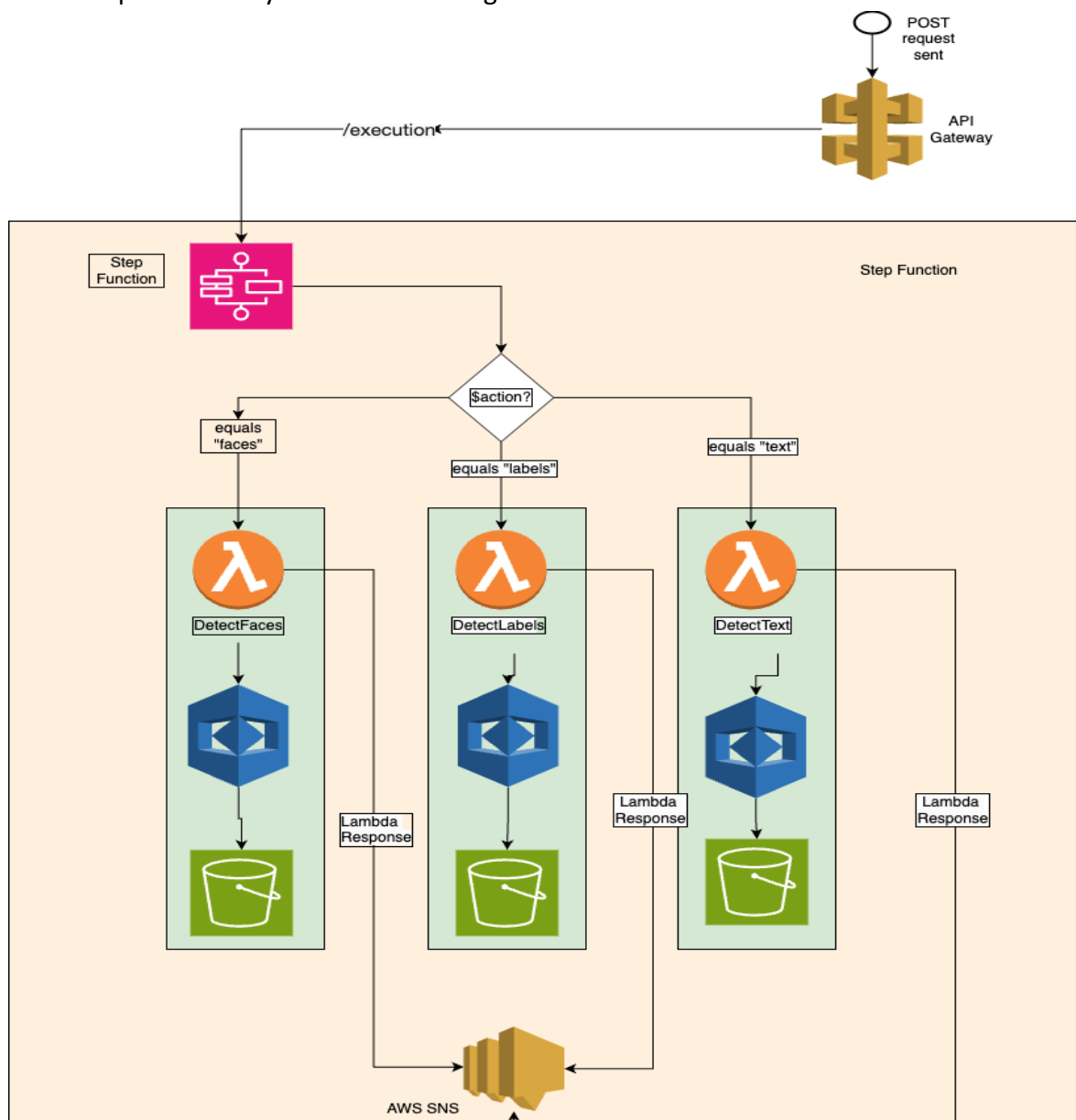
The delivery model I will be using is the **Function-as-a-Service**. My one function is the AWS Lambda Function that will trigger the AWS Rekognition service. Here are the reasons why I chose the FaaS delivery model:

- 1- **Focus mainly on code:** I am not interested in hosting my program on a specific machine, so I would not be interested to have full control over my hardware like in PaaS and IaaS. And unlike the SaaS delivery model, I am not interested in having a UI or managing a cloud service. With FaaS I just focus on the implementation of my AWS Lambda function and ensuring that it is triggered through a secure API Gateway, and that the event executes successfully.
- 2- **Pay-as-you-go pricing:** With FaaS, you only pay for the execution of functions, the number of invocations and the duration of execution. Which is more cost-effective than using any other delivery model, because they all make you pay for the services you are using (including servers).
- 3- **No Server Management:** Managing servers and infrastructure for that case could be very tedious and time consuming. With FaaS you don't have to worry about how your data is stored.
- 4- **Quick testing:** The "Test" feature allows code to be executed right after its written with given inputs, this is very helpful because it allows developers to deploy their code rapidly without having to spend time configuring the architecture like in IaaS. It also saves the time of "connecting" everything together, because now the code could be tested with the Test button in AWS Lambda.

Section 5: Architecture

My application a service-oriented architecture because all of the business functionalities are separated into discrete services that are defined and accessed independently. Then the services are combined together using the step function, so that they can talk to one another and have a smooth integration. The benefits of having a SOA architecture in comparison to other architectures is its enhanced flexibility, scalability, and maintainability while allowing each service to evolve independently. It also makes it easier to integrate new features or services in the future.

Here is a picture of my architecture design.



It is first triggered by sending a POST request to the “/execution” endpoint and this triggers the step function, which then triggers the AWS Lambda Function, and then the output of the function is broadcast to the users that are subscribed to the topic. The images that will be processed by AWS Rekognition are stored in an S3 bucket. (more details on the specific is in the first section)

I used python to implement my AWS Lambda functions, in this part of my application from within the Lambda function a call to AWS Rekognition’s public API is sent which includes the S3 bucket name and file name (image name), so that AWS Rekognition’s can analyze the picture and return the response of the analysis. The Lambda function then returns this analysis to the user via SNS. The Lambda functions were the only part of my application that required code (More details about the AWS Lambda implementation can be found in section 1).

For several reason, I believe python is the most suited programming language for the implementation of my lambda function. The first is its AWS SDK support, which allows it to have excellent integration with AWS services through the Boto3 library, as seen previously in the code for AWS Rekognition (Amazon, 2024). Next is the fact that AWS Lambda natively supports Lambda, which gives a seamless integration. Finally, python’s simplicity and readability which makes it a solid go-to, when choosing a programming language.

I deployed my system on the cloud by configuring and provisioning all the services independently, I then ensured that they work independently, and then I connected them to each other using the state machine.

Section 6: Security

Security is a shared responsibility between AWS and me according to the shared responsibility model (Amazon, 2024), most of the services that I use are very secure according to AWS. We will be going over the protocol and mechanism that AWS uses to ensure security in all its services, I will also go over how I ensured that the service was secure as my part of the shared responsibility.

Data in Transit

AWS Lambda: AWS ensures that the invocation, function processing, and returned data are encrypted using keys managed by AWS Key Management Service (KMS) (Amazon, 2024). I also ensured that IAM role given was labRole, this would be very important in a real life example, because the IAM permission specified could be a potential security vulnerability if given to the wrong individuals. This applies to all other services as well.

AWS Step Functions: Communications between Step Functions and other AWS services like Lambda using SSL/TLS and they require TLS 1.2 and recommend TLS 1.3. AWS recommends that confidential or sensitive information should never be placed into tags or free-form text fields such as a Name field. (Amazon, 2024).

AWS API Gateway: AWS ensure that the APIs created with Amazon API Gateway expose HTTPS endpoints only. API Gateway doesn't support unencrypted (HTTP) endpoints. Additionally, API Gateway manages the certificates for default execute-api endpoints, which is the endpoint that I have used when creating my API. Lastly, by adding only POST method to my API, this allows the used to only send POST requests to my API, anything else will be blocked. (Amazon, 2024).

AWS Rekognition: AWS Rekognition is the only service here that has data both at rest and in transit. We will consider the in transit here. Amazon Rekognition API endpoints only support secure connections over HTTPS. All communication is encrypted with Transport Layer Security (TLS) (Amazon, 2024).

AWS SNS: It is protected using IAM roles, which was discussed earlier, and for this I used the IAM role.

Data at Rest

AWS S3: maintains compliance programs, such as PCI-DSS, HIPAA/HITECH, FedRAMP, EU Data Protection Directive, and FISMA, to help you meet regulatory requirements.

I secure my S3 bucket by allowing it only to be accessible by AWS Rekognition. (Amazon, 2024).

AWS Rekognition: images passed to Amazon Rekognition API operations may be stored and used to improve the service. The stored images are encrypted at rest (Amazon S3) using AWS Key Management Service (SSE-KMS) (Amazon, 2024).

Vulnerability

Currently my biggest vulnerability is in my SNS, because I am allowing everyone to subscribe to the Topic, this is a huge problem, in the use case of the crime scene/policemen because I will want my data to remain confidential, I cannot, however, get around this in my term assignment because I do not have access to other AWS accounts to add in the AWS lambda function. However, in a real-life example, I will be specifying the necessary AWS accounts, and not allowing everyone to subscribe to my SNS Topic.

Finally, since users will be using this system to add photos to the S3 bucket, it is best security practice to use multi-factor authentication (MFA) with each account.

Section 7: Cost-Metrics

In this section I will be considering each resource independently, stating its up-front, on-going, and additional costs to build this system in the real world. Then I will give a price breakdown of the entire system with expected usage. Next, I will consider the most expensive feature, where it might be essential to monitor it closely to ensure that our costs don't spike up.

Pricing of System Components

For all of the services in my program there are no upfront costs, or additional costs, only ongoing costs based on usage. (In real-life business model, upfront costs could be considered if you will pay someone initially to configure and provision these services)

AWS Lambda: Charged per request and the duration of execution.

- Free tier: 1 million free requests per month and 3.2 million seconds of compute time per month (AWS, 2024).
- Paid tier: First 6 billion GB-seconds per month, you pay \$0.0000166667 for every GB-second, and you pay \$0.20 per 1M requests (AWS, 2024).

AWS Step Functions: There are 2 types of step functions, standard and express, in our state machine, we utilize a standard step function, aka does not change. Charged for the total number of state transitions across all your state machines, including retries.

- Free Tier: 4,000 free state transitions per month (AWS, 2024).
- Paid Tier: \$0.000025 per transitions (\$0.025 per 1,000 state transitions) (AWS, 2024).

AWS S3: There are several types of S3, for our service we will utilize S3 standard.

For Storing data:

- Free Tier: 5 GB in the S3 Standard storage class for 12 months (AWS, 2024).
- Paid Tier: First 50 TB / Month, is charged \$0.023 per GB, over 500 TB/Month is charged \$0.021 per GB (AWS, 2024).

For accessing data (GET)

- Free Tier: 20,000 Get Requests for 12 months (AWS, 2024).
- Paid Tier: \$0.0004 per 1000 request (AWS, 2024).

AWS API Gateway

- Free Tier: 1M REST API Calls Received | 1M HTTP API Calls Received | 1M messages | 750,000 connection minutes per month, for 12 months (AWS, 2024)
- Paid Tier: First 300 million \$1.00 per million, 300+ million you pay \$0.90 per million (AWS, 2024)

AWS Rekognition: Here we will be considering image pricing only, and utilizing group 2, which includes, detect faces, detect labels, and detect text.

- Free Tier: 5,000 images per month for free each, for 12 months (AWS, 2024).
- Paid Tier: First 1 million images \$0.001 per image, Next 4 million images \$0.0008 (AWS, 2024).

AWS SNS: Here we will consider the notification endpoint type as email.

- Free Tier: 1,000 notifications (AWS, 2024).
- Paid Tier: \$2.00 per 100,000 notifications (AWS, 2024).

Total System Price

I believe that this could be implemented using a totally free tier for small organizations. However, I will consider the cost breakdown of the analysis of 5 million image per month, and 5 users subscribed to the SNS topic. Additionally, it's important to note that our step function has 4 state transitions.

Here is the price break down for each service based on the prices mentioned above.

AWS Lambda: 5 million executions will cost \$0.80.

AWS Step Functions: For 5 million executions, each with 4 state transitions, the cost is approximately \$499.90

AWS S3: For storage and get for 5 million images, and 5 million GET requests is around \$54.84

AWS API Gateway: For 5 million API calls, the cost is \$5.00

AWS Rekognition: For processing 5 million images is \$4,196

AWS SNS: For sending 25 million notifications, the cost is \$500.00

The **total** price of deploying our system would be **5256.54** per month, and that is on a larger scale, where 5 million image per month is stored and accessed from an S3 bucket, processed using AWS Lambda and AWS Rekognition, and sent to 5 users each time an image is processed, (25 million notifications sent.)

As you can see based on our previous analysis the most expensive service out of all of the services is the **AWS Rekognition**. Therefore, it might be a good idea add additional monitoring using CloudWatch to ensure that it our prices do not surpass a predetermined budget, specified by our business requirement.

Section 9: Private Cloud

In order to reproduce my architecture in a private cloud, while providing relatively the same level of availability as my current cloud implementation we would need to implement many changes. We would need to add hardware, since my current implementation is 100% serverless, and does not require the provisioning of any hardware devices. Then we would need to consider the software that would run on top of the hardware. Finally, we will give a cost breakdown of the implementing this architecture. Most of the information that was taken to answer this section was extracted from Todd Robinson's article *Public vs Private Cloud: A Cost Tipping Point Guide for IT Professionals* (2024)

Hardware

We will need servers capable of running virtual machines to substitute for AWS Lambda. Additionally, to substitute S3 buckets we will need high-capacity, redundant storage solutions to store our image database (this needs to be big which will be very costly). For our network category we will need routers, switches and load balancers in order to manage traffic, in order to substitute for API Gateway. These are the basic building blocks for reproducing my application and making it functional. However, if we aim to achieve the level of reliability that AWS has, we will need backup systems to prepare for disaster management. We would also need to implement very expensive functionalities like cooling and physical security measures to protect and maintain our physical servers (similar to AWS data centers). However, we won't consider these extensive measures when we calculate our rough estimate for producing a private cloud model (Robinson, 2024).

Software

Regarding software the first thing we will require is virtual machines, to achieve virtualization. Next is the operation that will run the virtualization software and the application itself on the server. We will also need to database software like that will allow us to store and manage the data. Regarding security we will need firewall, anti-malware, intrusion detection/prevention systems, and many other security software to ensure our data and servers are secure (Robinson, 2024). We might need to purchase a software that analyzes processing to replace AWS Rekognition, which is the main component of our application.

Rough Estimate

The initial set up of the **hardware**, which includes servers, storage devices, and a secure network setup, would be around \$100,000 on a small scale. For purchasing the necessary software licenses of VMware, operating system like Windows, database software like Oracle, and finally a stack of security services, this would total up to \$50,000 annually.

The support and maintenance of the hardware and software would potentially total up 10,000 – 100,000 dollars annually (This is a huge range because maintenance can vary significantly unlike hardware and software's predetermined costs.)

Therefore, the **total** rough estimate would be \$150,000 of upfront costs, and around \$70,000 of annual costs for software and hardware maintenance. It's worth noting that this is the prices of the system while offering very minimal scalability, and no reliability or fault tolerance. Finally, when we compare it our total price that we calculated previously considering our public deployment model (**5256.54** per month), we see that it's a lot more cost-effective to deploy it using a public cloud deployment model, then a private cloud deployment model.

Section 10: Application Evolution

There is a lot of different ways that this application could evolve and continue its development, as mentioned in the beginning of the report, this is a very general application that could be applied to many use cases. The evolution of the application would be highly critical to the use case that it is being adapted to, below I will consider some specific and non-specific developments that could be used to enhance the application.

Since this is an image processing the system the most important component of it, in my opinion, is the actual analysis which builds on AWS Rekognition. Therefore, the best way moving forward is to utilize the custom labels that AWS Rekognition offers, of course this would be more expensive, but it will provide better solution for the users. Additionally, if the user requires even more specialized and extensive image processing then offered by AWS Rekognition's premium labels (Amazon, 2024), we could consider integrating Amazon SageMaker (Amazon, 2024) to train custom machine learning models for more specialized image to provide custom results to meet their requirements.

To enhance user's interaction with the system, and to avoid creating API calls, we could implement a UI which allows users to choose the image that they want to analyze from displayed pictures instead of directly accessing the S3 bucket and choose the action, could be implemented as a button, which determines the analysis that AWS Rekognition should return. This allows general users to be able to use our application, (no IT professionals needed) because it removes the need to send a POST request manually. The beauty of the service-oriented architecture is that to implement this change, to replace the API call, the only thing that would need to be changed is the removal of the API Gateway and the introduction of the completed UI. No other components would have to be changed, because all the components are separate, independent building blocks.

Another more specific enhancement that we could add to our application, is to make it highly secure, by investing in the expensive security protocols that AWS supports for extra charge. This would be necessary if the use-case is a legal case, or if the data being analyzed is very confidential and sensitive data. To implement a high level of security for users Amazon Cognito (Amazon, 2024) could be incorporated to manage user authentication and user data management. This approach would go hand in hand with implementing a UI for users, because it provides the front-end for general users to interface with the program using a chatbot.

References

- 1- AWS (2024) Amazon API Gateway pricing. Retrieved from: <https://aws.amazon.com/api-gateway/pricing/>
- 2- AWS (2024) Amazon Rekognition pricing. Retrieved from: <https://aws.amazon.com/rekognition/pricing/>
- 3- AWS (2024) Amazon SNS pricing. Retrieved from: <https://aws.amazon.com/sns/pricing/>
- 4- AWS (2024) Amazon S3 pricing. Retrieved from: <https://aws.amazon.com/s3/pricing/?p=pm&c=s3&z=4>
- 5- AWS (2024) Amazon Step Functions pricing. Retrieved from: https://aws.amazon.com/step-functions/pricing/#AWS_Step_Functions_Standard_Workflows_pricing_details
- 6- AWS (2024) Amazon Lambda pricing. Retrieved from: <https://aws.amazon.com/lambda/pricing/?p=pm&c=la&z=4>
- 7- Be a Better Dev (2020) SNS vs SQS Comparison? Whats the difference? | Learn with a practical example. Retrieved from: <https://www.youtube.com/watch?v=mXk0MNjlO7A>
- 8- Be a Better Dev (2023) How to Use Amazon Rekognition for Facial Recognition and Analysis. Retrieved from: <https://www.youtube.com/watch?v=3PGPfs-ARdo>
- 9- AWS (2024) Data encryption for AWS Rekognition Image. Retrived from: <https://docs.aws.amazon.com/rekognition/latest/dg/security-data-encryption.html#security-data-encryption-at-rest>
- 10- AWS (2024) Data encryption for AWS Step Function. Retrieved from: <https://docs.aws.amazon.com/step-functions/latest/dg/data-protection.html>)
- 11- AWS (2024) Data encryption for AWS API Gateway. Retrieved from: <https://docs.aws.amazon.com/apigateway/latest/developerguide/security.html>
- 12- Todd Robinson (Feb, 2024) Public vs Private Cloud: A Cost Tipping Point Guide for IT Professionals. [openmetal.com] Retrieved from: <https://openmetal.io/resources/blog/public-cloud-vs-private-cloud-cost-tipping-points/>

- 13- AWS (2024) Customer Identity And Access Management - Amazon Cognito. Retrieved from:
https://aws.amazon.com/pm/cognito/?gclid=Cj0KCQjwztOwBhD7ARIsAPDKnkA4bL3-xhT0FzRwPGjz4MBBkAdOo0bidZthJnsbYboUpu4GBfEAvBsaAuyfEALw_wcB&trk=7e6d84e1-c3f8-4068-8f2f-1d72efa9078e&sc_channel=ps&ef_id=Cj0KCQjwztOwBhD7ARIsAPDKnkA4bL3-xhT0FzRwPGjz4MBBkAdOo0bidZthJnsbYboUpu4GBfEAvBsaAuyfEALw_wcB:G:s&s_kwid=AL!4422!3!651612435560!e!!g!!amazon%20cognito!19828211238!146491690505
- 14- AWS (2024) Amazon SageMaker. Retrieved from: <https://aws.amazon.com/sagemaker/>